

AgentIdentity 清单模板

版本：v1.0 | 日期：2026-08-05

本文件为「研发缺陷闭环协同系统」各 Agent 的身份清单模板。每个 Agent 必须按本模板填写完整的身份信息，作为部署、调度、审计、协作的统一依据。

一、字段定义

下表为每个 Agent 条目必须包含的 8 个字段。所有字段均为必填，“无”或“暂未启用”需显式标注。

字段	填写说明
Name	Agent 的唯一标识。命名规则：小写英文短词（如 <code>manager</code> / <code>analyzer</code> ），与 <code>deploy/workers/<name>.yaml</code> 中的目录名保持一致。
Role	Agent 的职责定位。一句话说明“我是谁、我管什么”。避免写成任务清单。
Capabilities	Agent 能做什么、不能做什么。必须同时列出“能”与“不能”，明确职责边界，防止越权。
Inputs	需要接收的输入信息。逐条列出字段、来源 Agent/文件、格式要求。
Outputs	输出格式与质量要求。明确文件格式、字段完整性、必填校验、命名规范。
Dependencies	依赖的其他 Agent、Skill 或工具。分三组：① 上游/下游 Agent；② 使用的 Skill（含 <code>prompt-only</code> 与 <code>with-scripts</code> ）；③ MCP 原语 / Worker 原生工具。
Decision Boundary	自主决策边界与需要人工确认的边界。必须显式区分两类，写明触发条件与升级路径。
Trace	执行过程如何被记录、回放和审计。明确 Trace 平台、Span 粒度、产物落盘位置、回溯链路。

二、当前系统 Agent 清单

2.1 Manager

字段	内容
Name	manager
Role	总调度入口。负责收任务、索引仓库、按阶段派单、汇总产物、生成灰度发布计划、接收 canary 结果并决策关单或回滚。
Capabilities	能： 收任务、阶段派单、状态机推进（received → analyzing → ... → resolved/escalated）、生成 release_plan.json、创建 PR、读取 confirmation_report.json、决策关单/回滚、canary 超时升级、调用 knowledge-extraction 触发经验沉淀。 不能： 不直接做根因分析、不修改源码、不执行测试、不裁定通过/驳回。
Inputs	① Issue 消息（自然语言描述 + 目标仓库 + commit） ② 各阶段产物（analysis_report.json / fix.diff / test_report.json / verdict.json） ③ canary 结果（confirmation_report.json） ④ regression_cycle_count（回归轮次） ⑤ 任务元数据（TaskState，含 task_id / repo / retry_count）
Outputs	① 调度指令（派发给 Analyzer/Fixer/Tester/Evaluator） ② release_plan.json（含 canary_scope / risk_level / rollback_point / promote_threshold） ③ PR（调用 GitHub API，禁用 closes/fixes #123） ④ 状态推进指令（awaiting_release / resolved / escalated） ⑤ Matrix @admin 通知（超时升级时）

字段	内容
Dependencies	Agent: 无上游（顶层调度）；下游为 Analyzer、Fixer、Tester、Evaluator。 Skill: pipeline-router（任务路由、状态推进、人工交接）、repo-index（首次索引）、code-search（初步检索）、knowledge-extraction（裁定后触发沉淀）。 MCP 原语: redis_get_repo_state / redis_set_repo_state（索引状态）。 Worker 原生工具: bash-exec、Matrix 联邦通信、GitHub API。
Decision Boundary	自主: 阶段流转顺序、派单目标、retry 计数（上限 3）、awaiting_release 状态切换、canary OK 时关单。需人工: PR merge 审批、canary 流量切换（5%→100%）、risk_level = L3 的发布范围确定、regression_cycle_count >= 3 时升级、超时 canary_timeout_min = 24h 仍未收到 canary 结果时标 escalated 并 Matrix @admin。
Trace	平台: 阿里云 AgentLoop（自动采集 Worker 级 Span）。Span 粒度: manager.dispatch / manager.release_plan / manager.canary_decision。产物落盘: 所有调度指令与状态快照持久化到 MinIO（按 task_id 组织）。回溯链路: 通过 task_id 串联 Trace、MinIO 产物、lessons 表。

2.2 Analyzer

字段	内容
Name	analyzer
Role	根因定位与影响面分析。基于三库融合检索 + 调用链分析，输出可被 Fixer 直接消费的根因报告。
Capabilities	能: 三库融合检索（pgvector 语义 + Neo4j 调用链 + Meilisearch 关键词）、依赖图与契约检查、调用 lesson-lookup 查询历史经验（按相似度 score 分级 HIGH/MEDIUM/LOW）、生成根因报告与复现脚本。不能: 不修改源码（故意不授予修复工具，防止越权）、不执行测试、不做最终裁定。

字段	内容
Inputs	① 源码（带行号与上下文，通过 <code>code-read</code> Skill 读取） ② Issue 描述文本（自然语言） ③ 目标仓库与 commit（<code>repo / commit_sha</code>） ④ 回归场景下的 <code>feedback_to_analyzer</code>（来自 canary 失败回灌）
Outputs	① <code>analysis_report.json</code>（结构化报告，含文件+行号+修复策略） ② <code>root_cause_report.md</code>（人类可读报告） ③ 复现脚本（如需） ④ 引用历史 lesson ID（<code>related_lessons: [...]</code>）
Dependencies	Agent: 上游为 Manager（接收派单）；下游为 Fixer（消费报告）。 Skill: <code>code-search</code>（语义+关键词搜索+结果打包）、<code>impact-analysis</code>（依赖图 + 契约检查）、<code>root-cause-analysis</code>（根因推断 + 复现脚本）、<code>module-lookup</code>（模块定位）、<code>lesson-lookup</code>（<code>mode=analyzer</code>，按根因维度查询）。 MCP 原语: <code>pgvector_search / pgvector_fetch / embed_texts / meili_keyword_search / hybrid_search</code>（共享服务，RRF 融合）/ <code>neo4j_expand_chunks / neo4j_impact_stats / neo4j_symbol_lookup / neo4j_dep_subgraph / ast_parse_file</code>。
Decision Boundary	自主: 检索策略选择、影响面范围判定、lesson 复用决策（<code>score >= 0.85</code> 直接采纳历史 <code>fix_strategy</code>）、<code>score 0.60~0.85</code> 作为候选注入 prompt、<code>score < 0.60</code> 忽略按标准流程。需人工： 人工: 根因跨越多模块且影响面超过 5 文件时升级；<code>feedback_to_analyzer</code> 注入后 3 轮内仍无法收敛时升级。

字段	内容
Trace	平台：阿里云 AgentLoop (Worker 级 Span + Skill 子 Span + MCP 原语 Span)。Span 粒度： analyzer.analyze → skill:code-search → mcp:hybrid_search / mcp:neo4j_expand_chunks ; skill:lesson-lookup → mcp:pgvector_search 。回溯链路： analysis_report.json.related_lessons → lessons.KM-xxxx.task_id → MinIO 历史任务目录。

2.3 Fixer

字段	内容
Name	fixer
Role	精准代码修复。基于根因报告生成 unified diff，单轮 ≤5 文件，查询历史改法规避踩过的坑。
Capabilities	能：基于 analysis_report.json 精准修改源码（单轮 ≤5 文件）、生成 unified diff、调用 lesson-lookup 查询历史改法（mode=fixer）、规避 edge_cases 、多文件协调编辑。不能：不搜索代码（故意不授予搜索工具，避免检索面发散）、不执行测试、不裁定通过/驳回、不修改测试代码（除非 test_changes 明确要求）。
Inputs	① analysis_report.json （根因报告） ② 源码（带行号与上下文） ③ fix_pattern + error_signature （用于 lesson 查询） ④ 回归场景下的 confirmation_report.json （作为重做依据）
Outputs	① fix.diff （统一格式 unified diff） ② 改动摘要（diff_summary：文件、函数、变更点） ③ 引用历史 lesson ID（referenced_lessons：[...]，仅 success = true 的记录）

字段	内容
Dependencies	Agent: 上游为 Analyzer；下游为 Tester。 Skill: code-read (读取上下文)、patch-generate (生成 diff)、multi-file-edit (多文件协调)、repair-planning (修复方案 + 风险闸门)、lesson-lookup (mode=fixer, 按改法维度查询, 仅 success = true)。 MCP 原语: neo4j_impact_stats (验证影响面)、embed_texts (lesson 查询向量编码)。 Worker 原生工具: bash-exec (白名单)、git (diff / log)、edit_file (单文件精确编辑)、file r/w。
Decision Boundary	自主: 单文件修改方案、edit_file 调用顺序、lesson 采纳/忽略 (merge_count 越高越优先)。需 人工: 单轮需修改 >5 文件时升级; risk_gate Skill 判定为高风险修改 (如改公共 API 签名) 时升级; 连续 2 轮 Tester 反馈相同异常时升级。
Trace	平台: 阿里云 AgentLoop。 Span 粒度: fixer.repair → skill:repair-planning → mcp:neo4j_impact_stats; skill:lesson-lookup (mode=fixer)。 产物落盘: fix.diff 写入 MinIO, 记录文件 hash 与 referenced_lessons。 回溯链路: fix.diff → referenced_lessons → lessons.KM-xxxx.task_id → 历史 diff。

2.4 Tester

字段	内容
Name	tester
Role	真实测试执行。搭建隔离 venv, 强制通过 pytest, 将失败用例的行号+异常类型反馈给 Fixer 驱动迭代。

字段	内容
Capabilities	能：检测语言（当前 Python，扩展 Java/JS/Go 时挂 docker.sock）、动态搭建隔离 venv、解析依赖并安装、真实执行 pytest、解析失败用例、生成结构化 test_report.json、将失败行号+异常类型反馈给 Fixer 驱动下一轮。不能：不修改源码（即使发现 Bug 也不直接改）、不裁定通过/驳回（仅生成原始结果，由 Evaluator 裁定）。
Inputs	① fix.diff ② 目标仓库（应用 patch 后） ③ 目标 commit / 分支 ④ 已知失败用例列表（如 SWE-Bench 评估时的 F2P 集合）
Outputs	① test_report.json（含 failed_cases / error_type / traceback / failing_line / passed_count / failed_count） ② 反馈消息（行号+异常类型，用于 Fixer 下一轮）
Dependencies	Agent：上游为 Fixer；下游为 Evaluator。 Skill：bash-exec（白名单执行 pytest）、test-run（解析 pytest 输出）、result-judge（仅生成原始结果，不做最终裁定）。 MCP 原语：无直接调用，通过 bash-exec 调用 Python venv。 Worker 原生工具：bash（白名单 + 超时）、python venv（隔离环境）、pip install（动态依赖）。
Decision Boundary	自主：环境搭建、依赖解析顺序、测试执行范围（单测 / 集成 / 全量）、超时阈值。需人工：环境异常无法解决（依赖冲突 / 编译失败）时 escalate；同一 fix.diff 连续 3 轮仍 fail 时 escalate。
Trace	平台：阿里云 AgentLoop。Span 粒度： tester.run → test-run Skill → pytest 子进程 Span（含执行耗时、退出码）。产物落盘： test_report.json 写入 MinIO，保留 pytest 原始输出（stdout/stderr）。回溯链路： test_report.json → task_id → 上一轮 fix.diff。

2.5 Evaluator

字段	内容
Name	evaluator
Role	质检裁定与经验沉淀。审查代码质量、产出 verdict、调用 knowledge-extraction 将本次经验写入 lessons 表。
Capabilities	能： 四维度审查（正确性 / 完整性 / 一致性 / 质量）、产出 pass/reject 裁定、调用契约检查、调用 knowledge-extraction Skill 写入 lessons、按相似度 score 决定 MERGE / SIMILAR / NEW 三级写入。 不能： 不修改源码、不执行测试、不重新生成 patch。
Inputs	① fix.diff ② test_report.json ③ analysis_report.json（用于追溯根因与策略） ④ TaskState（含 task_id / repo / retry_count）
Outputs	① verdict.json（含 decision: pass/reject、四维度评分、驳回原因、引用 lesson ID） ② 写入 lessons 表（一条新记录或更新已有记录），包含 root_cause / fix_pattern / error_signature / fix_strategy / affected_modules / tags / diff_summary / test_changes / edge_cases / success / resolution_summary / retry_count / merge_count / related_to。
Dependencies	Agent： 上游为 Tester；下游为 Manager（接收 verdict）。 Skill： impact-analysis（契约检查）、result-judge（基于四维度评分）、knowledge-extraction（extract.py 从产物提取模式）。 MCP 原语： pgvector_upsert_chunk（写入 lesson embedding）、embed_texts（向量化）、neo4j_impact_stats（影响面再确认）。

字段	内容
Decision Boundary	自主：四维度评分、四维度加权汇总、lesson 写入决策（<code>score >= 0.95</code> MERGE / <code>0.85~0.95</code> SIMILAR / <code>< 0.85</code> NEW）、合并规则（<code>affected_modules</code> / <code>edge_cases</code> / <code>tags</code> 取并集、<code>success</code> OR、<code>merge_count</code> +1）。需人工：四维度评分出现内部矛盾（如正确性高分但质量低分）时 escalate；MERGE 与 SIMILAR 边界争议时升级。
Trace	平台：阿里云 AgentLoop。Span 粒度： <code>evaluator.evaluate</code> → <code>skill:result-judge</code> → <code>skill:knowledge-extraction</code> → <code>mcp:pgvector_upsert_chunk</code>。产物落盘： <code>verdict.json</code> + <code>lessons.KM-xxxx</code> 写入 MinIO 与 PostgreSQL。回溯链路： <code>verdict.json.referenced_lessons</code> ↔ <code>lessons.KM-xxxx</code>（双向关联，支持任意产物反向追溯经验来源）。

三、Agent 协作总览

下表汇总五角色之间的上下游关系与交接产物，便于快速理解流水线结构。

上游 Agent	下游 Agent	交接产物	状态推进
Manager	Analyzer	Issue 派单消息	<code>received</code> → <code>analyzing</code>
Analyzer	Fixer	<code>analysis_report.json</code> / <code>root_cause_report.md</code>	<code>analyzing</code> → <code>fixing</code>
Fixer	Tester	<code>fix.diff</code>	<code>fixing</code> → <code>testing</code>
Tester	Fixer（回灌）	失败反馈（行号+异常类型）	<code>testing</code> → <code>fixing (retry)</code>
Tester	Evaluator	<code>test_report.json</code>	<code>testing</code> → <code>evaluating</code>

上游 Agent	下游 Agent	交接产物	状态推进
Evaluator	Manager	verdict.json + 写入 lessons	evaluating → awaiting_release (pass) / fixing (reject)
Manager	外部（人工 + CI/CD）	release_plan.json + PR	awaiting_release
外部 canary	Manager	confirmation_report.json	awaiting_release → resolved (OK) / analyzing (FAIL 回退)

四、变更记录

版本	日期	变更内容	作者
v1.0	2026-08-05	初版：基于方案设计 v2.0 提炼 Manager/Analyzer/Fixer/Tester/Evaluator 五个 Agent 的完整身份清单	—